

LES DIFFÉRENTS TYPES DE BASES DE DONNÉES



INTRODUCTION

- Les bases de données sont au cœur des applications modernes.
 - Choisir le bon type est crucial pour la performance, la flexibilité et la scalabilité.
 - Objectifs :
 - Explorer les différents types de bases
 - Comparer leurs caractéristiques
 - Définir les critères de choix
- 

1 : BASES DE DONNÉES RELATIONNELLES (RDBMS)

- Organisation en **tables** (lignes/colonnes)
- Langage standard : **SQL**
- Très utilisées pour les applications classiques
- **Exemples** : MySQL, PostgreSQL, Oracle
- ✓ Avantages :
 - Cohérence, intégrité, transactions ACID
- ✗ Inconvénient :
 - Rigidité du schéma

2 : BASES NOSQL

a. Document

- Stockage sous forme de documents JSON/BSON
- **Exemple** : MongoDB
- ✓ Flexible

b. Clé-Valeur

- Données simples : clé + valeur
- **Exemple** : Redis
- ✓ Ultra rapide

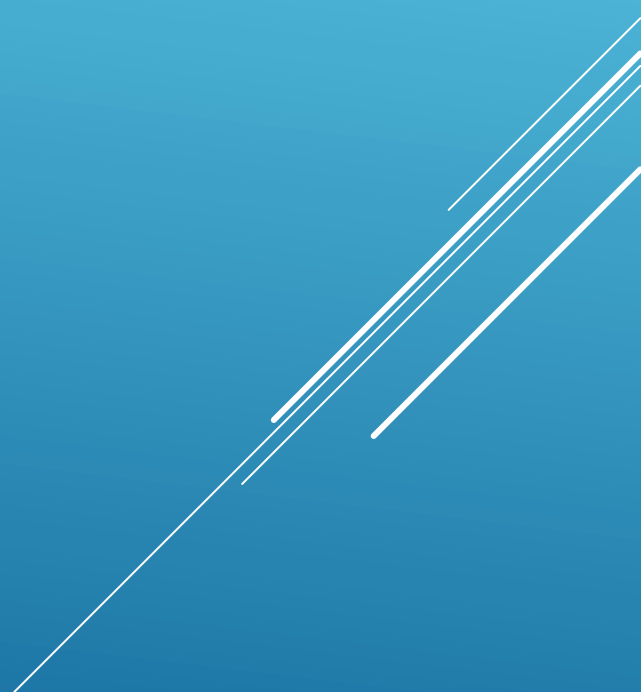
c. Colonnes

- Données réparties par colonnes
- **Exemple** : Cassandra
- ✓ Idéal pour le Big Data

d. Graphes

- Nœuds + relations
- **Exemple** : Neo4j
- ✓ Parfait pour les réseaux sociaux

3 : AUTRES TYPES

- **Orientée objets** : stocke des objets comme en POO (ex : db4o)
 - **Temporelle** : suit l'évolution dans le temps (ex : TimescaleDB)
 - **En mémoire** : rapide, stocke en RAM (ex : Redis, Memcached)
 - **Distribuée** : fonctionne sur plusieurs machines (ex : Cassandra)
 - **Cloud-native** : optimisée pour le cloud (ex : Firebase, Aurora)
- 
- Several white lines of varying lengths and slopes are positioned in the bottom right corner of the slide, creating a modern, abstract graphic element.

4 : TABLEAU COMPARATIF

Type	▼ Structure	▼ Cas d'usage	▼ Exemples	▼ Avantages	▼ Limites
Relationnelle	Tables	Apps classiques	PostgreSQL, MySQL	Cohérence, SQL, transactions	Moins flexible
NoSQL Document	JSON/BSON	Contenu, APIs	MongoDB	Flexible, schema-less	Requêtes complexes limitées
Clé-Valeur	clé : valeur	Cache, sessions	Redis, DynamoDB	Très rapide	Peu adapté aux relations
Colonnes	Familles de colonnes	Big Data	Cassandra, HBase	Scalabilité	Complexe à modéliser
Graphe	Nœuds + Arêtes	Réseaux, recommandations	Neo4j	Requêtes relationnelles puissantes	Moins courant
Temporelle	Timestamp + données	Monitoring, logs	InfluxDB, TimescaleDB	Optimisé pour les séries	Cas d'usage spécifique
En mémoire	RAM	Cache, calculs rapides	Redis, Memcached	Ultra performant	Données volatiles
Distribuée	Multi-nœuds	Tolérance aux pannes	Cassandra, Aurora	Haute disponibilité	Configuration complexe
Cloud-native	Variable	Apps cloud	Firebase, Spanner	Maintenance auto	Dépendance fournisseur

5 : CRITÈRES DE CHOIX

Critère	Impact sur le choix
Structure des données	Structurées → SQL ; Semi/non structurées → NoSQL
Volume & scalabilité	Gros volumes → NoSQL, colonnes, distribuées
Relations complexes	Oui → Graphes ou relationnel
Besoin de performance (vitesse)	Oui → En mémoire (Redis)
Historique / temps	Oui → Temporelle (TimescaleDB)
Environnement cloud	Oui → Cloud-native (Firebase, Aurora)
Facilité de développement	NoSQL + schema-less → rapide

Exemple 1 : Application e-commerce

- Produit → Document (MongoDB)
- Commandes → Relationnelle (PostgreSQL)
- Panier en cache → Redis

Exemple 2 : Réseau social

- Profils/relations → Graphe (Neo4j)
- Messages → Document (Couchbase)
- Monitoring → InfluxDB

6 : CONCLUSION

- Pas de “meilleure” base : le bon choix dépend du **contexte**.
 - Parfois, une **architecture hybride** est optimale.
 - Bien comprendre les types permet de faire des choix techniques solides.
- 
- Several white lines of varying lengths and orientations are positioned in the bottom right corner of the slide, creating a modern, abstract graphic element.

TP NOTÉ

Création d'une base de données

Exercice 1 : Création d'une base multilingue

- Créer une base multi_lang avec encodage UTF8 et collation française BUT3 de gestion des élèves de BUT3 et des notes dans différentes matières (Normalisée).
- Insérer des données avec accents et trier-les.

Gestion des utilisateurs et privilèges

Exercice 2 : Création d'utilisateurs

- Créer deux utilisateurs : admin_user (superutilisateur) et readonly_user (lecture seule).
- Leur attribuer un mot de passe.

Exercice 3 : Attribution de privilèges

- Créer dans la base une table Notes.
- Donner à readonly_user uniquement les droits de lecture sur la table.

Sauvegarde et restauration

Exercice 4 : Sauvegarde logique

- Utiliser pg_dump pour sauvegarder la base dans un fichier SQL.
- Restaurer cette sauvegarde dans une nouvelle base BUT3_backup.

Exercice 5 : Restauration ciblée

- Restaurer uniquement la table ayant la liste des étudiants dans une base vide à partir du fichier dump.